

PyTorch and Autograd

How one line computes every gradient · video 1:51

This lesson opens up the one line that does all the calculus in training: `loss.backward()`. PyTorch **records** every operation on a tensor as a graph, then walks that graph **in reverse** with the chain rule. One call fills in the gradient of every weight, and an optimizer uses those gradients to update the weights.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 9).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. Tensors that remember	▶ 0:08-0:31	3	☐
2. The forward pass builds a graph	▶ 0:31-0:45	3	☐
3. <code>backward()</code> : the chain rule in reverse	▶ 0:45-1:04	4	☐
4. One backward call, every gradient	▶ 1:04-1:15	4	☐
5. The optimizer, and <code>zero_grad</code>	▶ 1:15-1:30	4	☐
6. Autograd in code	▶ 1:30-1:37	2	☐

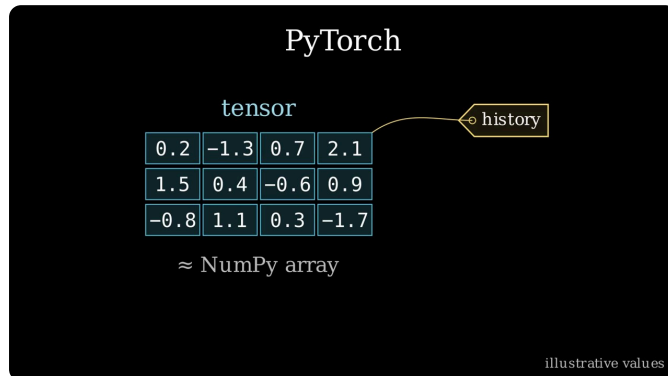
Before you start: this lesson builds on F10 (Slopes and Gradients: the derivative, the chain rule, gradient descent), F11 (NumPy arrays) and F12 (weights and training). It is the machinery behind the training loop of episode 11 (Training: Learning from Mistakes) and the tiny GPT of episode 12 (Build a Tiny GPT).

CONCEPT 1 OF 6

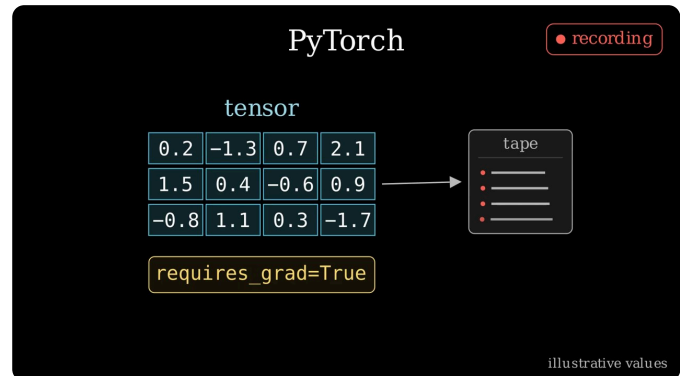
Tensors that remember

▶ Video 0:08-0:31

In episode 11, one line did all the calculus: `loss.backward()`. To see how, start with PyTorch's basic object, the **tensor**: an array of numbers, just like a NumPy array. The difference is that a tensor can **remember how it was computed**.



▶ 0:23 A tensor is an array of numbers, like a NumPy array, that can remember its history.



▶ 0:30 With `requires_grad=True`, PyTorch records every operation that uses the tensor.

Create a tensor with `requires_grad=True` and PyTorch starts **recording** every operation that uses it, like a tape. Later, that record is what lets PyTorch find gradients for you.

KEY IDEA

A tensor is a NumPy-like array that can keep a history. `requires_grad=True` switches the recording on.

Check 1: Tensors that remember

Answer all 3 questions before looking at the answers. Pass mark: all correct.

1.1 MULTIPLE CHOICE

For this lesson, what is the important difference between a PyTorch tensor and a NumPy array?

- A. A tensor can hold only one number
- B. A tensor can remember how it was computed, so its gradients can be found later
- C. A tensor is always two-dimensional
- D. NumPy arrays cannot do arithmetic

1.2 TRUE OR FALSE

"PyTorch records every operation on every tensor, whether or not `requires_grad=True` was set."

True ☐ False ☐

1.3 SHORT ANSWER

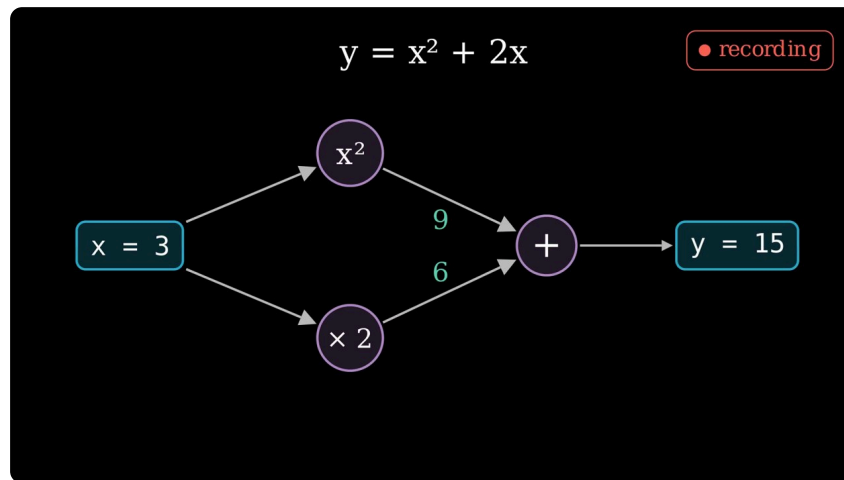
You create `a = torch.tensor(2.0)` and `b = torch.tensor(5.0, requires_grad=True)`, compute `c = a * b` and call `c.backward()`. (a) Which of a and b gets a gradient? (b) What is it?

CONCEPT 2 OF 6

The forward pass builds a graph

▶ Video 0:31-0:45

Take $x = 3$ and compute $y = x^2 + 2x$. As PyTorch computes, it builds a **graph**: x goes into a *square* (giving 9) and into a *times two* (giving 6), and the two results go into an *add*: $y = 9 + 6 = 15$.



▶ 0:45 $y = x^2 + 2x$ at $x = 3$: x goes into a square (9) and a times two (6); the add gives $y = 15$.

This is the **forward pass**. It gives the answer, 15, and as a side effect a record of how the answer was reached: one node per operation, with arrows showing which values feed which.

KEY IDEA

The forward pass computes the value **and** records a graph of the operations. Nothing about slopes has been computed yet.

Check 2: The forward pass builds a graph

Answer all 3 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

Same function $y = x^2 + 2x$, now with $x = 5$. What are the two middle values in the graph, and what is y ?

.....

2.2 SHORT ANSWER

Describe the graph PyTorch builds for $z = (x + 1)^2$, and the values in it for $x = 4$.

.....

2.3 TRUE OR FALSE

"The graph is built when you call `backward()`."

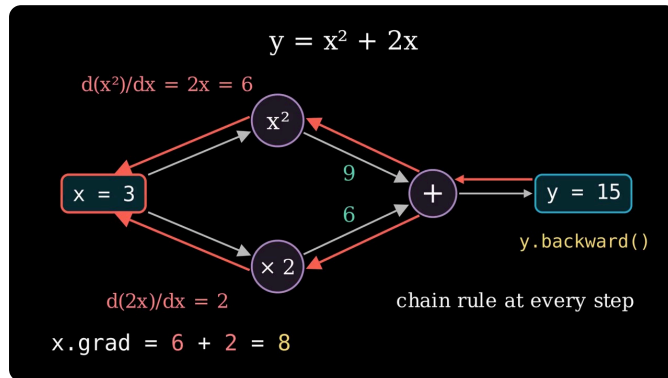
True ☐ False ☐

CONCEPT 3 OF 6

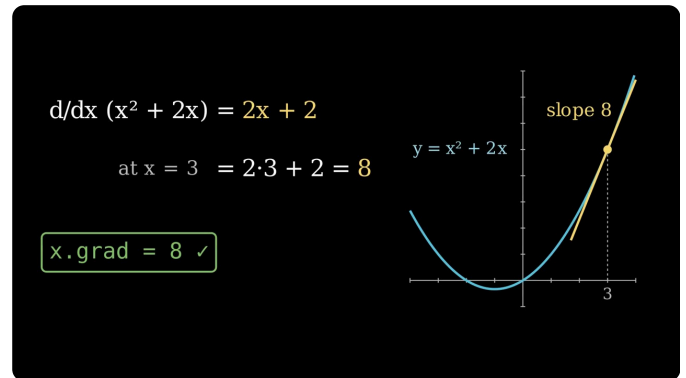
backward(): the chain rule in reverse

▶ Video 0:45–1:04

Call `y.backward()` and PyTorch walks the graph **in reverse**, applying the **chain rule** (F10) at every step. Each node knows its own slope: at $x = 3$ the square contributes $d(x^2)/dx = 2x = 6$, the times two contributes 2. Since x reaches y along two paths, the contributions **add up**: $6 + 2 = 8$. The result lands in `x.grad`.



▶ 0:54 `y.backward()` walks the graph in reverse. The two paths from x contribute 6 and 2: `x.grad = 8`.



▶ 1:03 The check: the slope of $x^2 + 2x$ is $2x + 2$, which is 8 at $x = 3$.

Check by hand: the derivative of $x^2 + 2x$ is $2x + 2$, and $2 \cdot 3 + 2 = 8$. Autograd gives exactly the slope you would find with calculus, without you writing any derivative.

KEY IDEA

`y.backward()` applies the chain rule automatically, from the output back to the inputs. The slope dy/dx is stored in `x.grad`.

Check 3: backward(): the chain rule in reverse

Answer all 4 questions before looking at the answers. Pass mark: all correct.

3.1 CALCULATION

$y = x^2 + 2x$ at $x = 5$. What will `x.grad` be after `y.backward()`?

3.2 CALCULATION

$z = (x + 1)^2$ at $x = 4$, as in question 2.2. What will `x.grad` be after `z.backward()`?

3.3 MULTIPLE CHOICE

In the video, why are 6 and 2 added to give `x.grad`?

- A. Because the order of operations says so
- B. Because x reaches y along two paths, and the effects of both paths add up
- C. Because PyTorch always adds 2 to a gradient
- D. Because `backward()` was called twice

3.4 TRUE OR FALSE

"To use autograd, you first have to work out the derivative formula (like $2x + 2$) yourself."

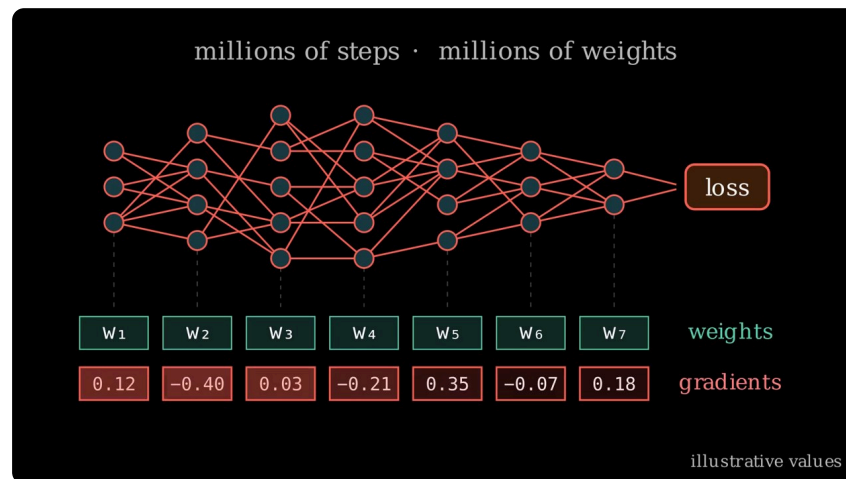
True ☐ False ☐

CONCEPT 4 OF 6

One backward call, every gradient

▶ Video 1:04-1:15

A real model is the same thing, just bigger. The forward pass from the inputs to the **loss** has millions of steps, and the loss depends on millions of weights. PyTorch records all of it, and **one** call to `loss.backward()` fills in the gradient of **every** weight: how the loss changes when that weight is nudged.



▶ 1:14 A real model: millions of steps up to the loss. One backward pass fills in a gradient for every weight (illustrative values).

KEY IDEA

However big the model, a single backward call gives one gradient per weight. This is backpropagation (F10), done for you.

Check 4: One backward call, every gradient

Answer all 4 questions before looking at the answers. Pass mark: all correct.

4.1 CALCULATION

The tiny GPT of episode 12 has 818,241 parameters. After one call to `loss.backward()`, how many gradient numbers have been filled in?

4.2 MULTIPLE CHOICE

What does the gradient of one weight tell you?

- A. The weight's current value
- B. How the loss changes when that weight is nudged a little, and in which direction
- C. How many times the weight was used
- D. The learning rate

4.3 SHORT ANSWER

In the figure, w_2 has gradient -0.40 and w_5 has gradient 0.35 . To lower the loss, should each weight go up or down?

4.4 TRUE OR FALSE

"A model with a million weights needs a million backward calls, one per weight."

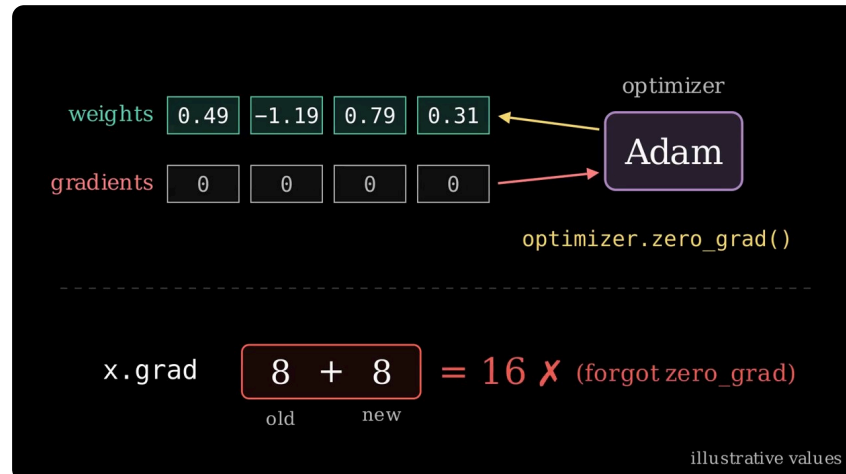
True ☐ False ☐

CONCEPT 5 OF 6

The optimizer, and zero_grad

▶ Video 1:15-1:30

Gradients alone change nothing. An **optimizer**, such as **Adam**, reads them and **updates the weights** (in the video $0.50 \rightarrow 0.49$, $-1.20 \rightarrow -1.19$, ...). Then `optimizer.zero_grad()` clears the gradients before the next backward pass. Why? PyTorch **adds** new gradients to whatever is already in `.grad`: call backward twice on $y = x^2 + 2x$ without clearing, and `x.grad` is $8 + 8 = 16$.



▶ 1:29 Adam reads the gradients and updates the weights; `zero_grad` clears the gradients. Forget it, and $8 + 8 = 16$.

KEY IDEA

`backward()` only fills in gradients; the optimizer's `step()` changes the weights. Clear old gradients with `zero_grad()` every step, or they pile up.

Check 5: The optimizer, and zero_grad

Answer all 4 questions before looking at the answers. Pass mark: all correct.

5.1 CALCULATION

$y = x^2 + 2x$ at $x = 3$. You compute y and call `y.backward()` three times in a row, never clearing. What is `x.grad`?

5.2 PUT IN ORDER

Put one training step in order: `optimizer.step()` · `loss.backward()` · `optimizer.zero_grad()` · *compute the loss (forward pass)*.

5.3 CALCULATION

Use plain gradient descent instead of Adam: $\text{new } x = x - 0.1 \times x.\text{grad}$, starting at $x = 3$. What is the new x with the correct gradient 8, and with the doubled 16?

5.4 TRUE OR FALSE

“`loss.backward()` changes the weights.”

True ☐ False ☐

CONCEPT 6 OF 6

Autograd in code

▶ Video 1:30-1:37

The whole idea fits in four lines: create a tensor that requires gradients, compute with it, call `backward()`, and read the gradient.

```
1 import torch
2
3 x = torch.tensor(3.0, requires_grad=True)
4 y = x ** 2 + 2 * x      # PyTorch records the graph
5 y.backward()            # chain rule, in reverse
6 x.grad                  # tensor(8.)
```

▶ 1:36 The code from the video: create, compute, backward, read.

```
import torch

x = torch.tensor(3.0, requires_grad=True)
y = x ** 2 + 2 * x      # PyTorch records the graph
y.backward()            # chain rule, in reverse
x.grad                  # tensor(8.)
```

KEY IDEA

Create with `requires_grad=True` → compute → `backward()` → read `.grad`.

Check 6: Autograd in code

Answer all 2 questions before looking at the answers. Pass mark: all correct.

6.1 MULTIPLE CHOICE

You leave out `requires_grad=True`: `x = torch.tensor(3.0)`, then the same two lines. What happens at `y.backward()`?

- A. `x.grad` becomes 8
- B. `x.grad` becomes 0
- C. An error: nothing was recorded, so there is no graph to walk back
- D. `x.grad` becomes 15

6.2 HANDS-ON CODE

Try it yourself. Now there are two tensors that require gradients, and the loop runs the same computation twice. (a) Predict the two printed lines, then run it. (b) Add one line so that both lines print the same gradients. (c) Change `x` to `-1.0`: what does the first line print?

```
import torch

x = torch.tensor(3.0, requires_grad=True)
w = torch.tensor(2.0, requires_grad=True)

for step in range(2):
    y = w * x ** 2 + 2 * x
    y.backward()
    print(step, y.item(), x.grad.item(), w.grad.item())
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. Tensors that remember · rewatch ▶ 0:08-0:31

- 1.1 B.** Both are arrays of numbers; only the tensor can record its history for autograd.
- 1.2 False.** Only tensors that require gradients (and results computed from them) are recorded. Plain tensors are just numbers.
- 1.3 (a) Only b. (b) `b.grad` = 2.** `a` was not recorded, so `a.grad` stays `None`. The slope of $a \times b$ with respect to b is $a = 2$.

2. The forward pass builds a graph · rewatch ▶ 0:31-0:45

- 2.1 25 and 10, so $y = 35$.** The square gives $5^2 = 25$, the times two gives 10, the add gives 35.
- 2.2 $x \rightarrow \text{add } 1 \rightarrow \text{square} \rightarrow z$; values 5, then 25.** Two operations in a chain this time: $4 + 1 = 5$, then $5^2 = 25$. Only one path runs from x to z .
- 2.3 False.** It is recorded during the forward computation. `backward()` walks the graph that is already there.

3. `backward()`: the chain rule in reverse · rewatch ▶ 0:45-1:04

- 3.1 12.** The paths contribute $2 \cdot 5 = 10$ and 2, so $10 + 2 = 12$. Check: $2x + 2 = 12$.
- 3.2 10.** Chain rule along the single path: the square contributes $2 \cdot 5 = 10$, the add 1 contributes 1, and $10 \times 1 = 10$.
- 3.3 B.** Nudging x changes both x^2 and $2x$, and y feels both changes.
- 3.4 False.** That is the point of autograd: the formula was only used to check the answer.

4. One backward call, every gradient · rewatch ▶ 1:04-1:15

- 4.1 818,241.** One gradient per parameter, all from the same single backward call.
- 4.2 B.** It is the slope of the loss with respect to that weight.
- 4.3 w_2 up, w_5 down.** Step against the slope (F10). A negative gradient means the loss falls as the weight rises.
- 4.4 False.** One backward pass walks the whole graph once and gives every gradient.

5. The optimizer, and `zero_grad` · rewatch ▶ 1:15-1:30

- 5.1 24.** Each call adds another 8: $8 + 8 + 8 = 24$.
- 5.2 `zero_grad` → forward → backward → step.** `zero_grad` only has to come before backward (the tiny GPT calls it right after the forward pass). `step` needs the fresh gradients, so it comes last.
- 5.3 2.2 with 8; 1.4 with 16.** $3 - 0.8 = 2.2$ and $3 - 1.6 = 1.4$. A forgotten `zero_grad` makes the step twice as big as it should be.
- 5.4 False.** It only fills in `.grad`. The optimizer's step changes the weights.

6. Autograd in code · rewatch ► 1:30-1:37

6.1 C. PyTorch raises a `RuntimeError` (“element 0 of tensors does not require grad and does not have a grad_fn”).

6.2 (a) 0 24.0 14.0 9.0 and 1 24.0 28.0 18.0 · (c) 0 0.0 -2.0 1.0 (a) $y = 2 \cdot 9 + 6 = 24$. The slope with respect to x is $2wx + 2 = 14$; with respect to w it is $x^2 = 9$. The second backward adds to the first, so the gradients double: 28 and 18. (b) At the end of the loop body add `x.grad.zero_()`; `w.grad.zero_()` (the job `optimizer.zero_grad()` does for every weight); both lines then print 24.0 14.0 9.0. (c) $y = 2 \cdot 1 - 2 = 0$, $x.\text{grad} = 2 \cdot 2 \cdot (-1) + 2 = -2$, $w.\text{grad} = (-1)^2 = 1$.